P2596R0
Improve std::hive::reshape

Published Proposal, 2022-06-10

Author:
Arthur O'Dwyer

Audience:
LEWG, LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:
8

Current Source:
github.com/Quuxplusone/draft/blob/gh-pages/d2596-improve-hive-reshape.bs

Current:
rawgit.com/Quuxplusone/draft/gh-pages/d2596-improve-hive-reshape.html

---

Abstract

P0447R20 `std::hive` proposes a complicated capacity model involving the library identifiers `hive_limits`, `block_capacity_limits`, `block_capacity_hard_limits`, `reshape`, in addition to `capacity`, `reserve`, `shrink_to_fit`, and `trim_capacity`. P0447R20's model permits the user to specify a max block size; this causes "secret quadratic behavior" on some common operations. P0447R20's model requires the container to track its min and max block sizes as part of its (non-salient) state.

We propose a simpler model that involves the library identifiers `max_block_size` and `reshape`, in addition to `capacity`, `reserve`, `shrink_to_fit`, and `trim_capacity`. This model does not permit the user to specify a max block size, so "secret quadratic behavior" is less common. This model does not require the container to track anything; the new `reshape` is a simple mutating operation analogous to `reserve` or `sort`.

Table of Contents

1. Changelog

- R0:
  - Initial release.

2. Introduction: P0447R20's model

P0447R20 `hive` is a bidirectional container, basically a "rope with holes." Compare it to the existing STL sequence containers:

- `vector` is a single block with spare capacity only on the end. Only one block ever exists at a time.
- `list` is a linked list of equal-sized blocks, each with capacity 1; unused blocks are immediately deallocated.
- `deque` is a vector of equal-sized blocks, each with capacity N; spare capacity exists at the end of the last block *and* at the beginning of the first block, but nowhere else.
- `hive` is a linked list of variably-sized blocks; spare capacity ("holes") can appear anywhere in any block, and the container keeps track of where the "holes" are.

The blocks of a `hive` are variably sized. The intent is that as you insert into a hive, it will allocate new blocks of progressively larger sizes — just like `vector`'s geometric resizing, but without relocating any existing elements. This improves cache locality when iterating.

We can represent a vector, list, or hive diagrammatically, using `[square brackets]` to bracket the elements belonging to a single block, and `_` to represent spare capacity ("holes"). There's a lot of bookkeeping detail not captured by this representation; that's okay for our purposes.

```
std::vector<int> v = {1,2,3,4,5,6}; // [1 2 3 4 5 6 _ _]
std::list<int>   l = {1,2,3,4,5,6}; // [1] [2] [3] [4] [5] [6]
std::hive<int>   h = {1,2,3,4,5,6}; // [1 2 3 4 5 6]
```

Erasing from a vector shifts the later elements down. Erasing from a list or hive never needs to shift elements.

```
v.erase(std::next(v.begin()));  // [1 3 4 5 6 _ _ _]
l.erase(std::next(l.begin()));  // [1] [3] [4] [5] [6]
h.erase(std::next(h.begin()));  // [1 _ 3 4 5 6]
```

Reserving in a vector may invalidate iterators, because there's no way to strictly "add" capacity. Reserving in a hive never invalidates iterators (except for `end()`), because we can just add new blocks right onto the back of the container. (In practice, `hive` tracks unused blocks in a separate list so that iteration doesn't have to traverse them; this diagrammatic representation doesn't capture that detail.)

```
v.reserve(10);  // [1 3 4 5 6 _ _ _ _ _]
h.reserve(10);  // [1 _ 3 4 5 6] [_ _ _ _]
```

P0447R20 allows the programmer to specify `min` and `max` block capacities, via the `std::hive_limits` mechanism. No block in the hive is ever permitted to be smaller than `min` elements in capacity, nor greater than `max` elements in capacity. For example, we can construct a hive in which no block's capacity will ever be smaller than 4 or greater than 5.

```
std::hive<int> h({1,2,3,4,5,6}, std::hive_limits(4, 5));
               // [1 2 3 4 5] [6 _ _ _]
h.reserve(10); // [1 2 3 4 5] [6 _ _ _] [_ _ _ _]
```

A hive can also be "reshaped" on the fly, after construction. In the following example, after creating a hive with one size-5 block, we `reshape` the hive to include only blocks of sizes between 3 and 4 inclusive. This means that the size-5 block is now "illegal"; its elements are redistributed to other blocks, and then it is deallocated, because this hive is no longer allowed to hold blocks of that size.

```
std::hive<int> h({1,2,3,4,5,6}, std::hive_limits(4, 5));
               // [1 2 3 4 5] [6 _ _ _]
h.reserve(10); // [1 2 3 4 5] [6 _ _ _] [_ _ _ _]
h.reshape(std::hive_limits(3, 4));
               // [6 1 2 3] [4 5 _ _]
```

Notice that `reshape` invalidates iterators (to element `1`, for example), and can also undo the effects of `reserve` by reducing the overall capacity of the hive. (Before the reshape operation,

`h.capacity()` was 13; afterward it is only 8.) Programmers are advised to "`reshape` first, `reserve` second."

§ 3. Criticisms of P0447R20's model

§ 3.1. Max block size is not useful in practice

One of the primary motivations for `hive` is to be usable in embedded/low-latency situations, where the programmer might want fine control over the memory allocation scheme. So at first glance it makes sense that the programmer should be able to specify min and max block capacities via `hive_limits`. However:

- A programmer is likely to care about *min* block capacity (for cache locality), but not so much *max* capacity. The more contiguity the better! Why would I want to put a cap on it?
- If the embedded programmer cares about max capacity, it's likely because they're using a slab allocator that hands out blocks of some fixed size (say, 1024 bytes). But that doesn't correspond to `hive_limits(0, 1024)`. The min and max values in `hive_limits` are *counts of elements*, not the size of the actual allocation. So you might try dividing by `sizeof(T)`; but that still won't help, for two reasons:
  - Just like with `std::set`, the size of the allocation is not the same as `sizeof(T)`. In the reference implementation, a block with capacity `n` typically asks the allocator for `n*sizeof(T) + n + 1` bytes of storage, to account for the skipfield structure. In my own implementation, I do a `make_shared`-like optimization that asks the allocator for `sizeof(GroupHeader) + n*sizeof(T) + n + 1` bytes.
  - The reference implementation doesn't store `T` objects contiguously, when `T` is small. When `plf::hive<char>` allocates a block of capacity `n`, it actually asks for `n*2 + n + 1` bytes instead of `n*sizeof(char) + n + 1` bytes.

It's kind of fun that you're allowed to set a very small maximum block size, and thus a hive can be used to simulate a traditional "rope" of fixed-capacity blocks:

```
std::hive<int> h(std::hive_limits(3, 3));
h.assign({1,2,3,4,5,6,7,8});  // [1 2 3] [4 5 6] [7 8 _]
```

It's kind of fun; but I claim that it is not *useful enough* to justify its cost, in brain cells nor in CPU time.

Imagine if `std::vector` provided this max-block-capacity API!

```
// If vector provided hive's API...
std::vector<int> v = {1,2};  // [1 2]
v.reshape({5, 8});           // [1 2 _ _ _]
v.assign({1,2,3,4,5});       // [1 2 3 4 5]
v.push_back(6);              // [1 2 3 4 5 6 _ _]
v.push_back(7);              // [1 2 3 4 5 6 7 _]
v.push_back(8);              // [1 2 3 4 5 6 7 8]
v.push_back(9);              // throws length_error, since allocating a lar
```

No, the real-world `vector` sensibly says that it should just keep geometrically resizing until the underlying allocator conks out. In my opinion, `hive` should behave the same way. Let the *allocator* decide how many bytes is too much to allocate at once. Don't make it the container's problem.

> Note: Discussion in SG14 (2022-06-08) suggested that maybe `hive_limits(16, 16)` would be useful for SIMD processing; but that doesn't hold up well under scrutiny. The reference implementation doesn't store `T` objects contiguously. And even if `hive_limits(16, 16)` were useful (which it's not), that rationale wouldn't apply to `hive_limits(23, 29)` and so on.

3.2. Max block size causes O(n) behavior

3.2.1. Example 1

> Note: The quadratic behavior shown below was *previously* present in Matt Bentley's reference implementation (as late as git commit 9486262), but was fixed in git commit 7ac1f03 by renumbering the blocks less often.

Consider this program parameterized on `N`:

```
std::hive<int> h(std::hive_limits(3, 3));
h.assign(N, 1); // [1 1 1] [1 1 1] [1 1 1] [...
while (!h.empty()) {
    h.erase(h.begin());
}
```

This loop takes O(N^2) time to run! The reason is that `hive`'s active blocks are stored in a linked list, but also *numbered* in sequential order starting from 1; those "serial numbers" are required by `hive::iterator`'s `operator<`. (Aside: I don't think `operator<` should exist either, but my understanding is that that's already been litigated.)

Every time you `erase` the last element from a block (changing `[_ _ 1]` into `[_ _ _]`), you need to move the newly unused block from the active block list to the unused block list. And then you need to renumber all the subsequent blocks. Quoting P0447R20's specification of `erase`:

> *Complexity:* Constant if the active block within which `position` is located does not become empty as a result of this function call. If it does become empty, at worst linear in the number of subsequent blocks in the iterative sequence.

Since this program sets the max block capacity to 3, it will hit the linear-time case once for every three erasures. Result: quadratic running time.

3.2.2. Example 2

> Note: The quadratic behavior shown below is *currently* present in Matt Bentley's reference implementation, as of git commit ef9bad9.

Consider this program parameterized on `N`:

```
plf::hive<int> h;
h.reshape({4, 4});
for (int i=0; i < N; ++i) {
    h.insert(1);
    h.insert(2);
}
erase(h, 1); // [_ 2 _ 2] [_ 2 _ 2] [_ 2 _ 2] [...
while (!h.empty()) {
    h.erase(h.begin());
}
```

The final loop takes O(N^2) time to run! The reason is that in the reference implementation, `hive`'s list of blocks-with-erasures is singly linked. Every time you `erase` the last element from a block (changing `[_ _ _ 2]` into `[_ _ _ _]`), you need to unlink the newly empty block from the list of blocks with erasures; and thanks to how we set up this snippet, the current block will always happen to

be at the end of that list! So each `erase` takes O(`h.size()`) time, and the overall program takes O(N^2) time.

Such "secret quadratic behavior" is caused *primarily* by how `hive` permits the programmer to set a max block size. If we get rid of the max block size, then the implementation is free to allocate larger blocks, and so we'll hit the linear-time cases geometrically less often — we'll get amortized O(`N`) instead of O(N^2).

> Using my proposed `hive`, it will still be possible to simulate a rope of fixed-size blocks; but the programmer will have to go pretty far out of their way. There will be no footgun analogous to `std::hive<int>({1,2,3}, {3,3})`.
>
> ```
> auto make_block = []() {
>     std::hive<int> h;
>     h.reserve(3);
>     return h;
> };
> std::hive<int> h;
> h.splice(make_block()); // [_ _ _]
> h.splice(make_block()); // [_ _ _] [_ _ _]
> h.splice(make_block()); // [_ _ _] [_ _ _] [_ _ _]
> // ...
> ```

§ 3.3. Move semantics are arguably unintuitive

Suppose we've constructed a hive with min and max block capacities, and then we assign into it from another sequence in various ways.

```
std::hive<int> h(std::hive_limits(3, 3));  // empty

h.assign({1,2,3,4,5});                // [1 2 3] [4 5 _]
h = {1,2,3,4,5};                      // [1 2 3] [4 5 _]
h.assign_range(std::hive{1,2,3,4,5}); // [1 2 3] [4 5 _]

std::hive<int> d = h;                 // [1 2 3] [4 5 _]
std::hive<int> d = std::move(h);      // [1 2 3] [4 5 _]
std::hive<int> d(h, Alloc());            // [1 2 3] [4 5 _]
std::hive<int> d(std::move(h), Alloc()); // [1 2 3] [4 5 _]
```

```
// BUT:

h = std::hive{1,2,3,4,5}; // [1 2 3 4 5]
// OR
std::hive<int> h2 = {1,2,3,4,5};
h = h2;                   // [1 2 3 4 5]
// OR
std::hive<int> h2 = {1,2,3,4,5};
std::swap(h, h2);         // [1 2 3 4 5]

// BUT AGAIN:

h = std::hive<int>(std::hive_limits(3, 3));
h.splice({1,2,3,4,5});  // throws length_error
```

The *`current-limits`* of a hive are not part of its "value," yet they still affect its behavior in critical ways. Worse, the capacity limits are "sticky" in a way that reminds me of `std::pmr` allocators: *most* modifying operations don't affect a hive's limits (resp. a `pmr` container's allocator), but *some* operations do.

The distinction between these two overloads of `operator=` is particularly awkward:

```
h = {1,2,3,4,5};           // does NOT affect h's limits
h = std::hive{1,2,3,4,5};  // DOES affect h's limits
```

§ 3.4. `splice` is O(n), and can throw

In P0447R20, `h.splice(h2)` is a "sticky" operation: it does not change `h`'s limits. This means that if `h2` contains any active blocks larger than `h.block_capacity_limits().max`, then `h.splice(h2)` will fail and throw `length_error`! This is a problem on three levels:

- Specification speedbump: Should we say something about the state of `h2` after the throw? for example, should we guarantee that any too-large blocks not transferred out of `h2` will remain in `h2`, kind of like what happens in `std::set::merge`? Or should we leave it unspecified?
- Correctness pitfall: `splice` "should" just twiddle a few pointers. The idea that it might actually *fail* is not likely to occur to the working programmer.
- Performance pessimization: Again, `splice` "should" just twiddle a few pointers. But P0447R20's specification requires us to traverse `h2` looking for too-large active blocks. This adds an O(n) step that doesn't "need" to be there.

If my proposal is adopted, `hive::splice` will be "Throws: Nothing," just like `list::splice`.

> Note: I would expect that both `hive::splice` and `list::splice` *ought* to throw `length_error` if the resulting container would exceed `max_size()` in size; but I guess that's intended to be impossible in practice.

§ 3.5. `hive_limits` causes constructor overload set bloat

Every STL container's constructor overload set is "twice as big as necessary" because of the duplication between `container(Args...)` and `container(Args..., Alloc)`. `hive`'s constructor overload set is "four times as big as necessary" because of the duplication between `container(Args...)` and `container(Args..., hive_limits)`.

P0447R20 `hive` has 18 constructor overloads. My proposal removes 7 of them. (Of course, we could always eliminate these same 7 constructor overloads without doing anything else to P0447R20. If this were the *only* complaint, my proposal would be undermotivated.)

Analogously: Today there is no constructor overload for `vector` that sets the capacity in one step; it's a multi-step process. Even for P0447R20 `hive`, there's no constructor overload that sets the overall capacity in one step — even though overall capacity is more important to the average programmer than min and max block capacities!

```
// Today's multi-step process for vector
std::vector<int> v;
v.reserve(12);            // [_ _ _ _ _ _ _ _ _ _ _ _]
v.assign({1,2,3,4,5,6});  // [1 2 3 4 5 6 _ _ _ _ _ _]

// Today's multi-step process for hive
std::hive<int> h;
h.reshape(std::hive_limits(12, h.block_capacity_hard_limits().max));
h.reserve(12);            // [_ _ _ _ _ _ _ _ _ _ _ _]
h.reshape(h.block_capacity_hard_limits());
h.assign({1,2,3,4,5,6});  // [1 2 3 4 5 6 _ _ _ _ _ _]

// Today's (insufficient) single-step process for hive
// fails to provide a setting for overall capacity
std::hive<int> h({1,2,3,4,5,6}, {3, 5});
                          // [1 2 3 4 5] [6 _ _]
```

If my proposal is adopted, the analogous multi-step process will be:

```
std::hive<int> h;
h.reshape(12, 12);      // [_ _ _ _ _ _ _ _ _ _ _ _]
h.assign({1,2,3,4,5,6}; // [1 2 3 4 5 6 _ _ _ _ _ _]
```

§ 3.6. `hive_limits` introduces unnecessary UB

[D0447R20] currently says ([hive.overview]/5):

> If user-specified limits are supplied to a function which are not within an implementation's *hard limits*, or if the user-specified minimum is larger than the user-specified maximum capacity, the behaviour is undefined.

In Matt Bentley's reference implementation, this program will manifest its UB by throwing `std::length_error`.

The following program's behavior is always undefined:

```
std::hive<int> h;
h.reshape({0, SIZE_MAX}); // UB! Throws.
```

Worse, the following program's definedness hinges on whether the user-provided max `1000` is greater than the implementation-defined `hive<char>::block_capacity_hard_limits().max`. The reference implementation's `hive<char>::block_capacity_hard_limits().max` is `255`, so this program has undefined behavior (Godbolt):

```
std::hive<char> h;
h.reshape({10, 1000}); // UB! Throws.
```

There are two problems here. The first is trivial to solve: P0447R20 adds to the set of unnecessary library UB. We could fix that by simply saying that the implementation must *clamp the provided limits* to the hard limits; this will make `h.reshape({0, SIZE_MAX})` well-defined, and make `h.reshape({10, 1000})` UB only in the unlikely case that the implementation doesn't support *any* block sizes in the range [10, 1000]. We could even mandate that the implementation *must* throw `length_error`, instead of having UB.

The second problem is bigger: `hive_limits` vastly increases the "specification surface area" of `hive`'s API.

- Every constructor needs to specify (or UB) what happens when the supplied `hive_limits` are invalid.
- `reshape` needs to specify (or UB) what happens when the supplied `hive_limits` are invalid.
- `splice` needs to specify what happens when the two hives' *`current-limits`* are incompatible.
- The special members — `hive(const hive&)`, `hive(hive&&)`, `operator=(const hive&)`, `operator=(hive&&)`, `swap` — need to specify their effect on each operand's *`current-limits`*. For example, is `operator=(const hive&)` permitted to preserve its left-hand side's *`current-limits`*, in the same way that `vector::operator=(const vector&)` is permitted to preserve the left-hand side's capacity? The Standard needs to specify this.
- `reshape` needs to think about how to restore the hive's invariants if an exception is thrown. (See below.)

All this extra specification effort is costly, for LWG and for vendors. My "Proposed Wording" is mostly deletions.

When I say "`reshape` needs to think about how to restore the hive's invariants," I'm talking about this example:

```
std::hive<W> h({1,2,3,4,5}, {4,4}); // [1 2 3 4] [5 _ _ _]
h.reshape({5,5});  // suppose W(W&&) can throw
```

Suppose `W`'s move constructor is throwing (and for the sake of simplicity, assume `W` is move-only, although the same problem exists for copyable types too). The hive needs to get from `[1 2 3 4] [5 _ _ _]` to `[1 2 3 4 5]`. We can start by allocating a block `[_ _ _ _ _]` and then moving the first element over: `[1 2 3 4] [5 _ _ _ _]`. Then we move over the next element, intending to get to `[1 2 3 _] [5 4 _ _ _]`; but that move construction might throw. If it does, then we have no good options. If we do as `vector` does, and simply deallocate the new buffer, then we'll lose data (namely element `5`). If we keep the new buffer, then we must update the hive's *`current-limits`* because a hive with limits `{4,4}` cannot hold a block of capacity 5. But a hive with the new user-provided limits `{5,5}` cannot hold a block of capacity 4! So we must either lose data, or replace `h`'s *`current-limits`* with something "consistent but novel" such as `{4,5}` or `h.block_capacity_hard_limits()`. In short: May a failed operation leave the hive with an "out-of-thin-air" *`current-limits`*? Implementors must grapple with this kind of question in many places.

§ 3.7. Un-criticism: Batches of input/output

The `reshape` and `hive_limits` features are not mentioned in any user-submitted issues on [PlfColony], but there is one relevant user-submitted issue on [PlfStack]:

> If you know that your data is coming in groups of let's say 100, and then another 100, and then 100 again etc but you don't know when it is gonna stop. Having the size of the group set at 100 would allow to allocate the right amount of memory each time.

In other words, this programmer wants to do something like this:

```
// OldSmart
int fake_input[100] = {};
std::hive<int> h;
h.reshape(100, 100);  // blocks of size exactly 100
while (true) {
    if (rand() % 2) {
        h.insert(fake_input, fake_input+100);  // read a whole block
    } else {
        h.erase(h.begin(), std::next(h.begin(), 100));  // erase and "unuse
    }
}
```

If the programmer doesn't call `reshape`, we'll end up with `h.capacity() == 2*h.size()` in the worst case, and a single big block being used roughly like a circular buffer. This is actually not bad.

A programmer who desperately wants the exactly-100 behavior can still get it, after this patch, by working only a tiny bit harder:

```
// NewSmart
int fake_input[100] = {};
std::hive<int> h;
while (true) {
    if (rand() % 2) {
        if (h.capacity() == h.size()) {
            std::hive<int> temp;
            temp.reserve(100);
            h.splice(temp);
        }
        h.insert(fake_input, fake_input+100);  // read a whole block
```

```
    } else {
        h.erase(h.begin(), std::next(h.begin(), 100));  // erase and "unuse
    }
}
```

I have benchmarked these options ([Bench]) and see the following results. (`Matt` is Matthew Bentley's original `plf::hive`, `Old` is my implementation of P0447, `New` is my implementation of P2596; `Naive` simply omits the call to `reshape`, `Smart` ensures all blocks are size-`N` using the corresponding approach above.) With N=100, no significant difference between the `Smart` and `Naive` versions:

```
$ bin/ubench --benchmark_display_aggregates_only=true --benchmark_repetitic
2022-06-10T00:32:11-04:00
Running bin/ubench
Run on (8 X 2200 MHz CPU s)
CPU Caches:
  L1 Data 32 KiB (x4)
  L1 Instruction 32 KiB (x4)
  L2 Unified 256 KiB (x4)
  L3 Unified 6144 KiB (x1)
Load Average: 2.28, 2.14, 2.06
BM_PlfStackIssue1_MattSmart_mean        93298 ns        93173 ns
BM_PlfStackIssue1_MattNaive_mean        93623 ns        93511 ns
BM_PlfStackIssue1_OldSmart_mean        114478 ns       114282 ns
BM_PlfStackIssue1_OldNaive_mean        113285 ns       113124 ns
BM_PlfStackIssue1_NewSmart_mean        128535 ns       128330 ns
BM_PlfStackIssue1_NewNaive_mean        116530 ns       116380 ns
```

With N=1000 (`Matt` can't handle this because `h.block_capacity_hard_limits().max < 1000`), again no significant difference:

```
$ bin/ubench --benchmark_display_aggregates_only=true --benchmark_repetitic
2022-06-10T00:33:12-04:00
Running bin/ubench
Run on (8 X 2200 MHz CPU s)
CPU Caches:
  L1 Data 32 KiB (x4)
  L1 Instruction 32 KiB (x4)
  L2 Unified 256 KiB (x4)
```

```
  L3 Unified 6144 KiB (x1)
Load Average: 2.13, 2.17, 2.08
BM_PlfStackIssue1_OldSmart_mean       973149 ns       972004 ns
BM_PlfStackIssue1_OldNaive_mean       970737 ns       969565 ns
BM_PlfStackIssue1_NewSmart_mean      1032303 ns      1031035 ns
BM_PlfStackIssue1_NewNaive_mean      1011931 ns      1010680 ns
```

§ 4. New model

In my proposed new capacity model, `hive_limits` is gone. Shape becomes an ephemeral property of a `hive` object, just like capacity is an ephemeral property of a `vector` or `deque`. Other ephemeral properties of `hive` include capacity and sortedness:

```
std::hive<int> h = {1,2,3};
h.erase(h.begin());  // [_ 2 3]
h.sort();            // [_ 2 3]
h.insert(4);         // [4 2 3], no longer sorted

h.reshape(4, 8);     // [4 2 3 _] [_ _ _ _]
auto h2 = h;         // [4 2 3], no longer capacious
```

I propose the interface `bool reshape(size_t min, size_t n=0)`, where `min` is the new minimum block size (that is, the new hive's elements are guaranteed at least *x* degree of cache-friendliness) and `n` is the new capacity (that is, you're guaranteed to be able to insert *x* more elements before touching the allocator again). Note that `h.reshape(0, n)` means the same thing as `h.reserve(n)` except that `reshape` is allowed to invalidate iterators.

- The `bool` return value tells the programmer whether iterator invalidation occurred. I think this is likely to be useful information; if it is not provided, then the programmer *must always assume* that iterator invalidation has taken place.
- `min` is the argument that affects the hive's shape (so it is non-optional); `n` has a sensible default. If you call `h.reshape(min)` without providing a new capacity `n`, then you're saying you don't care about the capacity of the hive. One-argument `reshape` is very much like `shrink_to_fit`.
- Taking two adjacent `size_t` arguments is unfortunate. But notice that they are in the "correct" order: if you flipped them around, like `reshape(n, min=0)`, then `h.reshape(x)` would do exactly the same thing as `h.reserve(x)`, and that would be silly. So we have a good mnemonic for why the `min` argument should come first.

- There is no longer any way to force a max block size. If the implementation wants to make your elements "more contiguous," you can't stop it (fixing § 3.2 Max block size causes O(n) behavior).
- Block sizes are ephemeral, not "sticky"; they needn't be stored in data members (fixing § 3.3 Move semantics are arguably unintuitive).

This new model has been implemented in a fork of Matt Bentley's implementation; see [Impl]. Compile with `-DPLF_HIVE_P2596=0` for P0447R20, or `-DPLF_HIVE_P2596=1` for this new model.

§ 5. Proposed wording relative to P0447R20

The wording in this section is relative to [D0447R20] as it stands today.

> In addition to these changes, every reference in P0447 to "reserved blocks" should be changed to "unused blocks" for clarity. The vast majority of those references can simply be deleted, because my proposal largely eliminates the normative distinction between "active" and "unused" blocks. For example, P0447R20's specification for `erase` currently says
>
> > If the active block which `position` is located within becomes empty [...] as a result of the function call, that active block is either deallocated or transformed into a reserved block.
>
> After my proposal, that sentence can be removed, because it doesn't carry normative weight anymore: sure, the container will still behave exactly that way, but we no longer need to normatively *specify* that the empty block is non-active, because we no longer need to normatively prevent it from interfering with a `splice`. (After this patch, `splice` can never fail to transfer a block for any reason, so it doesn't need to go out of its way to avoid transferring unused blocks, so we don't need to normatively describe the tracking of unused blocks anymore. The separate unused-block list remains the intended implementation technique, for performance; but it will no longer be directly observable by the programmer.)

§ 5.1. [hive.overview]

1. A hive is a sequence container that allows constant-time insert and erase operations. Insertion position is not specified, but will in most implementations typically be the back of the container when no erasures have occurred, or when erasures have occurred it will reuse existing erased element memory locations. Storage management is handled automatically and is specifically organized in multiple blocks of sequential elements.

2. Erasures use unspecified techniques to mark erased elements as skippable, as opposed to relocating subsequent elements during erasure as is expected in a vector or deque. These elements are subsequently skipped during iteration. If a memory block becomes empty of unskipped elements as the result of an erasure, that memory block is removed from the iterative sequence. The same, or different unspecified techniques may be used to record the locations of erased elements, such that those locations may be reused later during insertions.

3. Operations pertaining to the updating of any data associated with the erased-element skipping mechanism or erased-element location-recording mechanism are not factored into individual function time complexity. The time complexity of these unspecified techniques is implementation-defined and may be constant, linear or otherwise defined.

4. Memory block element capacities have an unspecified growth factor greater than 1, for example a new block's capacity could be equal to the summed capacities of the existing blocks.

5. ~~Limits can be placed on the minimum and maximum element capacities of memory blocks, both by a user and by an implementation. In neither case shall minimum capacity be greater than maximum capacity. When limits are not specified by a user, the implementation's default limits are used. The default limits of an implementation are not guaranteed to be the same as the minimum and maximum possible values for an implementation's limits. The latter are defined as hard limits. If user-specified limits are supplied to a function which are not within an implementation's hard limits, or if the user-specified minimum is larger than the user-specified maximum capacity, behaviour is undefined.~~

6. Memory blocks can be removed from the iterative sequence [Example: by `erase` or `clear` —end example] without being deallocated. Other memory blocks can be allocated without becoming part of the iterative sequence [Example: by `reserve` —end example]. These are both referred to as *reserved blocks.* Blocks which form part of the iterative sequence of the container are referred to as *active blocks.*

7. A hive conforms to the requirements for Containers, with the exception of operators `==`, `!=`, and `<=>`. A hive also meets the requirements of a reversible container, of an allocator-aware container, and some of the requirements of a sequence container, including several of the optional sequence container requirements. Descriptions are provided here only for operations on `hive` that are not described in that table or for operations where there is additional semantic information.

8. Hive iterators meet the Cpp17BidirectionalIterator requirements but also provide relational operators `<`, `<=`, `>`, `>=`, and `<=>`, which compare the relative ordering of two iterators in the sequence of a hive instance.

```
namespace std {
```

```
struct hive_limits {
  size_t min;
  size_t max;
  constexpr hive_limits(size_t minimum, size_t maximum) noexcept : min(mini
};

template<class T, class Allocator = allocator<T>>
class hive {
private:
  hive_limits current-limits = implementation-defined; // exposition only
public:
  // types
  using value_type = T;
  using allocator_type = Allocator;
  using pointer = typename allocator_traits<Allocator>::pointer;
  using const_pointer = typename allocator_traits<Allocator>::const_pointer
  using reference = value_type&;
  using const_reference = const value_type&;
  using size_type = implementation-defined; // see [container.requirements]
  using difference_type = implementation-defined; // see [container.require
  using iterator = implementation-defined; // see [container.requirements]
  using const_iterator = implementation-defined; // see [container.requirem
  using reverse_iterator = std::reverse_iterator<iterator>; // see [contain
  using const_reverse_iterator = std::reverse_iterator<const_iterator>; //

  constexpr hive() noexcept(noexcept(Allocator())) : hive(Allocator()) { }
  explicit hive(const Allocator&) noexcept;
  explicit hive(hive_limits block_limits) noexcept(noexcept(Allocator())) :
  hive(hive_limits block_limits, const Allocator&) noexcept;
  explicit hive(size_type n, const Allocator& = Allocator());
  explicit hive(size_type n, hive_limits block_limits, const Allocator& = A
  hive(size_type n, const T& value, const Allocator& = Allocator());
  hive(size_type n, const T& value, hive_limits block_limits, const Allocat
  template<class InputIterator>
    hive(InputIterator first, InputIterator last, const Allocator& = Alloca
  template<class InputIterator>
    hive(InputIterator first, InputIterator last, hive_limits block_limits,
  template<container-compatible-range<T> R>
    hive(from_range_t, R&& rg, const Allocator& = Allocator());
  template<container-compatible-range<T> R>
    hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& =
  hive(const hive&);
  hive(hive&&) noexcept;
```

```
  hive(const hive&, const type_identity_t<Allocator>&);
  hive(hive&&, const type_identity_t<Allocator>&);
  hive(initializer_list<T>, const Allocator& = Allocator());
  hive(initializer_list<T>, hive_limits block_limits, const Allocator& = Al
  ~hive();
  hive& operator=(const hive& x);
  hive& operator=(hive&& x) noexcept(allocator_traits<Allocator>::propagate
  hive& operator=(initializer_list<T>);
  template<class InputIterator>
    void assign(InputIterator first, InputIterator last);
  template<container-compatible-range<T> R>
    void assign_range(R&& rg);

  void assign(size_type n, const T& t);
  void assign(initializer_list<T>);
  allocator_type get_allocator() const noexcept;

  // iterators
  iterator               begin() noexcept;
  const_iterator         begin() const noexcept;
  iterator               end() noexcept;
  const_iterator         end() const noexcept;
  reverse_iterator       rbegin() noexcept;
  const_reverse_iterator rbegin() const noexcept;
  reverse_iterator       rend() noexcept;
  const_reverse_iterator rend() const noexcept;

  const_iterator         cbegin() const noexcept;
  const_iterator         cend() const noexcept;
  const_reverse_iterator crbegin() const noexcept;
  const_reverse_iterator crend() const noexcept;

  // capacity
  [[nodiscard]] bool empty() const noexcept;
  size_type size() const noexcept;
  size_type max_size() const noexcept;
  size_type max_block_size() const noexcept;
  size_type capacity() const noexcept;
  void reserve(size_type n);
  bool reshape(size_t min, size_t n = 0);
  void shrink_to_fit();
  void trim_capacity() noexcept;
  void trim_capacity(size_type n) noexcept;
```

```
  void trim_capacity(size_type n = 0) noexcept;

  // modifiers
  template<class... Args> iterator emplace(Args&&... args);
  iterator insert(const T& x);
  iterator insert(T&& x);
  void insert(size_type n, const T& x);
  template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
  template<container-compatible-range<T> R>
    void insert_range(R&& rg);
  void insert(initializer_list<T>);
  iterator erase(const_iterator position);
  iterator erase(const_iterator first, const_iterator last);
  void swap(hive&) noexcept(allocator_traits<Allocator>::propagate_on_conta
  void clear() noexcept;

  // hive operations
  void splice(hive& x);
  void splice(hive&& x);
  size_type unique();
  template<class BinaryPredicate>
    size_type unique(BinaryPredicate binary_pred);

  hive_limits block_capacity_limits() const noexcept;
  static constexpr hive_limits block_capacity_hard_limits() noexcept;
  void reshape(hive_limits block_limits);

  iterator get_iterator(const_pointer p) noexcept;
  const_iterator get_iterator(const_pointer p) const noexcept;
  bool is_active(const_iterator it) const noexcept;

  void sort();
  template<class Compare> void sort(Compare comp);
}

template<class InputIterator, class Allocator = allocator<iter-value-type<I
  hive(InputIterator, InputIterator, Allocator = Allocator())
    -> hive<iter-value-type<InputIterator>, Allocator>;

template<class InputIterator, class Allocator = allocator<iter-value-type<I
  hive(InputIterator, InputIterator, hive_limits block_limits, Allocator =
    -> hive<iter-value-type<InputIterator>, block_limits, Allocator>;
```

```
template<ranges::input_range R, class Allocator = allocator<ranges::range_v
  hive(from_range_t, R&&, Allocator = Allocator())
    -> hive<ranges::range_value_t<R>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_v
  hive(from_range_t, R&&, hive_limits block_limits, Allocator = Allocator()
    -> hive<ranges::range_value_t<R>, block_limits, Allocator>;
}
```

An incomplete type `T` may be used when instantiating `hive` if the allocator meets the allocator completeness requirements ([allocator.requirements.completeness]). `T` shall be complete before any member of the resulting specialization of `hive` is referenced.

§ 5.2. [hive.cons]

```
explicit hive(const Allocator&) noexcept;
```

Effects: Constructs an empty hive, using the specified allocator.

Complexity: Constant.

```
hive(hive_limits block_limits, const Allocator&) noexcept;
```

Effects: Constructs an empty hive, with the specified Allocator. Initializes *`current_limits`* with `block_limits`.

Complexity: Constant.

```
explicit hive(size_type n, const Allocator& = Allocator());
hive(size_type n, hive_limits block_limits, const Allocator& = Allocator())
```

Preconditions: `T` is *Cpp17DefaultInsertable* into `hive`.

Effects: Constructs a hive with `n` default-inserted elements, using the specified allocator. If the second overload is called, also initializes *`current_limits`* with `block_limits`.

Complexity: Linear in `n`. ~~Creates at most `(n / current_limits.max) + 1` element block allocations.~~

```
hive(size_type n, const T& value, const Allocator& = Allocator());
hive(size_type n, const T& value, hive_limits block_limits, const Allocator
```

Preconditions: `T` is *Cpp17CopyInsertable* into `hive`.

Effects: Constructs a hive with `n` copies of value, using the specified allocator. ~~If the second overload is called, also initializes *`current_limits`* with `block_limits`.~~

Complexity: Linear in `n`. ~~Creates at most `(n / current_limits.max) + 1` element block allocations.~~

```
template<container-compatible-range<T> R>
  hive(from_range_t, R&& rg, const Allocator& = Allocator());
template<container-compatible-range<T> R>
  hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& = A
```

Preconditions: `T` is *Cpp17EmplaceConstructible* into `hive` from `*ranges::begin(rg)`.

Effects: Constructs a hive object with the elements of the range `rg`. ~~If the second overload is called, also initializes *`current_limits`* with `block_limits`.~~

Complexity: Linear in `ranges::distance(rg)`. ~~Creates at most `(ranges::distance(rg) / current_limits.max) + 1` element block allocations.~~

```
hive(initializer_list<T> il, const Allocator& = Allocator());
hive(initializer_list<T> il, hive_limits block_limits, const Allocator& = A
```

Preconditions: `T` is *Cpp17CopyInsertable* into `hive`.

Effects: Constructs a hive object with the elements of `il`. ~~If the second overload is called, also initializes *`current_limits`* with `block_limits`.~~

Complexity: Linear in `il.size()`. ~~Creates at most `(il.size() / current_limits.max) + 1` element block allocations.~~

§ 5.3. [hive.capacity]

§ 5.3.1. max_block_size

```
size_type max_block_size() const noexcept;
```

Returns: The largest possible capacity of a single memory block.

Complexity: Constant.

§ 5.3.2. capacity

```
size_type capacity() const noexcept;
```

Returns: The total number of elements that the hive can hold without requiring allocation of more ~~element memory~~ blocks.

Complexity: Constant time.

§ 5.3.3. reserve

```
void reserve(size_type n);
```

Effects: A directive that informs a hive of a planned change in size, so that it can manage the storage allocation accordingly. ~~Does not cause reallocation of elements. Iterators~~ Invalidates the past-the-end iterator. Iterators and references to elements in `*this` remain valid. If `n <= capacity()` there are no effects.

~~*Complexity:* It does not change the size of the sequence and creates at most `(n / block_capacity_limits().max) + 1` element block allocations.~~

Throws: `length_error` if `n > max_size()`. Any exception thrown from `allocate`.

Postconditions: `capacity() >= n` is `true`.

5.3.4. shrink_to_fit

```
void shrink_to_fit();
```

Preconditions: `T` is *Cpp17MoveInsertable* into `hive`.

Effects: `shrink_to_fit` is a non-binding request to reduce `capacity()` to be closer to `size()`. [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations. *—end note*] It does not increase `capacity()`, but may reduce `capacity()`. ~~It may reallocate elements.~~ Invalidates all references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. [*Note:* This operation may change the iterative order of the elements in `*this`. *—end note*]

Complexity: ~~If reallocation happens, linear in the size of the sequence.~~ Linear in `size()`.

~~*Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. [*Note:* If no reallocation happens, they remain valid. *—end note*] [*Note:* This operation may change the iterative order of the elements in *this. *—end note*]~~

5.3.5. trim_capacity

```
void trim_capacity() noexcept;
void trim_capacity(size_type n) noexcept;
void trim_capacity(size_type n = 0) noexcept;
```

Effects: ~~Removes and deallocates reserved blocks created by prior calls to `reserve`, `clear`, or `erase`. If such blocks are present, for the first overload `capacity()` is reduced. For the second overload `capacity()` will be reduced to no less than `n`.~~ `trim_capacity` is a non-binding request to reduce `capacity()` to be closer to `n`. It does not increase `capacity()`; it may reduce `capacity()`, but not below `n`.

Complexity: Linear in the number of reserved blocks ~~deallocated~~ .

Remarks: ~~Does not reallocate elements and no~~ No iterators or references to elements in `*this` are invalidated.

§ 5.4. [hive.operations]

In this subclause, arguments for a template parameter named `Predicate` or `BinaryPredicate` shall meet the corresponding requirements in [algorithms.requirements]. The semantics of `i + n` and `i - n`, where `i` is an iterator into the list and `n` is an integer, are the same as those of `next(i, n)` and `prev(i, n)`, respectively. For `sort`, the definitions and requirements in [alg.sorting] apply.

~~`hive` provides a splice operation that destructively moves all elements from one hive to another. The behavior of splice operations is undefined if `get_allocator() != x.get_allocator()`.~~

§ 5.4.1. splice

```
void splice(hive& x);
void splice(hive&& x);
```

Preconditions: `addressof(x) != this` is `true`.

Effects: Inserts the contents of `x` into `*this` and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements ~~shall~~ continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

Complexity: ~~At worst, linear in the number of active blocks in `x` + the number of active blocks in `*this`.~~ Linear in the number of active blocks in the resulting hive.

Throws: ~~`length_error` if any of `x`'s element memory block capacities are outside of the current minimum and maximum element memory block capacity limits of `*this`.~~ Nothing.

Remarks: The behavior ~~of splice operations~~ is undefined if `get_allocator() != x.get_allocator()`. ~~Reserved blocks in `x` are not transferred into `*this`.~~

§ 5.4.2. block_capacity_limits

```
hive_limits block_capacity_limits() const noexcept;
```

~~*Effects:* Returns `current_limits`.~~

~~*Complexity:* Constant.~~

5.4.3. block_capacity_hard_limits

```
static constexpr hive_limits block_capacity_hard_limits();
```

Returns: A `hive_limits` struct with the `min` and `max` members set to the implementation's hard limits.

Complexity: Constant.

5.4.4. reshape

```
void reshape(hive_limits block_limits);
bool reshape(size_t min, size_t n = 0);
```

Preconditions: `T` shall be *Cpp17MoveInsertable* into `hive`.

Effects: Sets minimum and maximum element memory block capacities to the `min` and `max` members of the supplied `hive_limits` struct. If the hive is not empty, adjusts existing memory block capacities to conform to the new minimum and maximum block capacities, where necessary. If existing memory block capacities are within the supplied minimum/maximum range, no reallocation of elements takes place. If they are not within the supplied range, elements are reallocated to new or existing memory blocks which fit within the supplied range, and the old memory blocks are deallocated. If elements are reallocated, all iterators and references to reallocated elements are invalidated. Reallocates storage by allocating new blocks and/or transferring elements among existing blocks so that the capacity of each memory block that remains allocated is greater than or equal to `min`, and `capacity()` is greater than or equal to `n`. If reallocation takes place, all iterators, pointers, and references to elements of the container, including the past-the-end iterator, are invalidated. Reallocation may change the iteration order of the elements of `*this`.

Returns: `true` if any iterators were invalidated; otherwise `false`.

Complexity: At worst linear in the number of active and reserved blocks in `*this`. If reallocation occurs, also linear in the number of elements reallocated. Linear in `size()`.

Throws: `length_error` if `min > max_block_size() || n > max_size()`. Any exception thrown from `allocate` or from the initialization of `T`. If reallocation occurs, uses `Allocator::allocate()` which may throw an appropriate exception. [*Note:* This operation may change the iterative order of the elements in `*this`. — *end note*]

Postconditions: `capacity() >= n` is `true`.

§ 6. Acknowledgements

Thanks to Matthew Bentley for proposing `std::hive`.

Thanks to Joe Gottman for his comments on a draft of this paper.

§ References

§ Informative References

[Bench]
Arthur O'Dwyer. *Benchmark code (git branch 'ajo')*. June 2022. URL: https://github.com/Quuxplusone/SG14/blob/e0012e6b56/benchmarks/hive_bench.cpp

[D0447R20]
Matthew Bentley. *Introduction of std::hive to the standard library*. May 2022. URL: https://plflib.org/D0447R20_-_Introduction_of_hive_to_the_Standard_Library.html

[Impl]
Arthur O'Dwyer. *Reference implementation of P2596R0 (git branch 'ajo')*. June 2022. URL: https://github.com/Quuxplusone/SG14/blob/e0012e6b56/include/sg14/plf_hive.h

[PlfColony]
Matthew Bentley. *plf_colony: All Issues*. August 2016–April 2022. URL: https://github.com/mattreecebentley/plf_colony/issues?q=is%3Aissue

[PlfStack]
Etienne Parmentier; Matthew Bentley. *plf_stack: change_group_sizes and related functions not available*. November 2020. URL: https://github.com/mattreecebentley/plf_stack/issues/1